04 常用 UI 元件

黃彬華編撰

- Slider 滑動改值
- Progress Indicator 進度指標
- Alert Dialog 對話視窗
- Dialog 自訂對話視窗
- DatePicker 日期挑選器
- DatePickerDialog 日期挑選視窗
- TimePicker 時間挑選器
- Switch 開關選項
- Checkbox 多選功能
- RadioButton 單選功能
- ExposedDropdownMenu 下拉選單
- Scaffold 版面配置容器
- 表格
- 列表
- 下拉更新
- 列表搜尋
- 列表的增刪改查
- MVC 模式
- MVC 走樣了？
- MVVM 模式
- MVVM 各元件說明

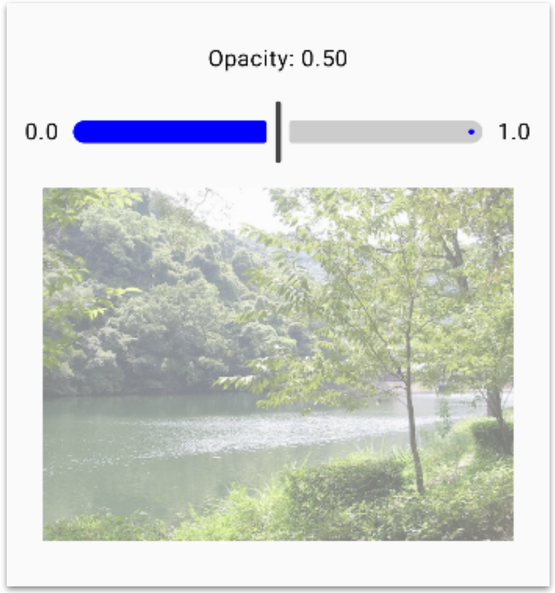
Slider 滑動改值

黃彬華編撰

- 以水平滑動方式改變值
- 呼叫 Slider() 建立，重要參數如下
  - value: 目前的值
  - valueRange: 值的範圍
  - onValueChanged: 值改變時的 callback
  - colors: 設定顏色
    - thumbColor: 設定 thumb 顏色
    - activeTrackColor: 設定 thumb 左方值域顏色
    - inactiveTrackColor: 設定 thumb 右方值域顏色

範例 SliderDemo

黃彬華編撰



- 滑動 Slider 會顯示值，而且會改變圖片透明度

Ex 4.1

黃彬華編撰



- 準備 4 個 Slider，讓使用者分別指定 "Text" 文字的大小與顏色 (紅、綠、藍)

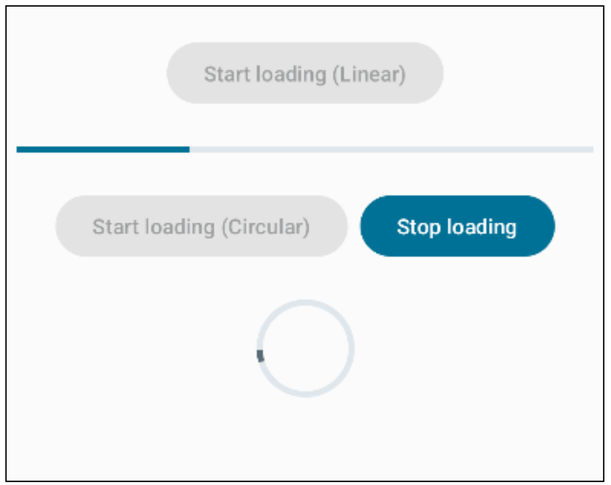
Progress Indicator 進度指標

黃彬華編撰

- 進度指標顯示需要等待的圖示，有 2 種樣式
  - 線形 (LinearProgressIndicator)
    - 具體告訴使用者目前進度
    - 呼叫 LinearProgressIndicator() 建立，重要參數如下
      - progress: 進度，介於 0.0 ~ 1.0
  - 圓形 (CircularProgressIndicator)
    - 使用者需要等待，但不確定要等多久
    - 呼叫 CircularProgressIndicator() 建立，重要參數如下
      - color: indicator 的顏色
      - trackColor: indicator 後面的軌道顏色

範例 ProgressIndicatorDemo

黃彬華編撰



- 由上至下分別展示線形與圓形進度指標

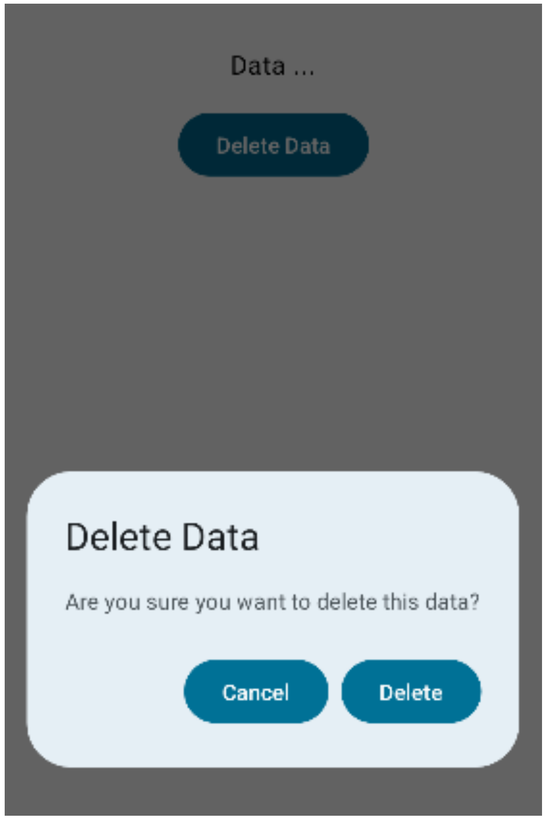
Alert Dialog 對話視窗

黃彬華編撰

- AlertDialog 會顯示對話訊息與對應操作按鈕，屬於簡易、制式化的對話視窗
- 呼叫 AlertDialog() 建立，重要參數如下
  - onDismissRequest: 點擊對話視窗外部或 back 按鈕時呼叫，並非點擊 dismissButton 時呼叫
  - title: 標題文字
  - text: 訊息文字
  - confirmButton: 設定確定按鈕
  - dismissButton: 設定取消按鈕

範例 AlertDialogDemo

黃彬華編撰



- 點擊 "Delete Data" 按鈕會跳出 AlertDialog
- 點擊 AlertDialog 上的按鈕會顯示對應訊息

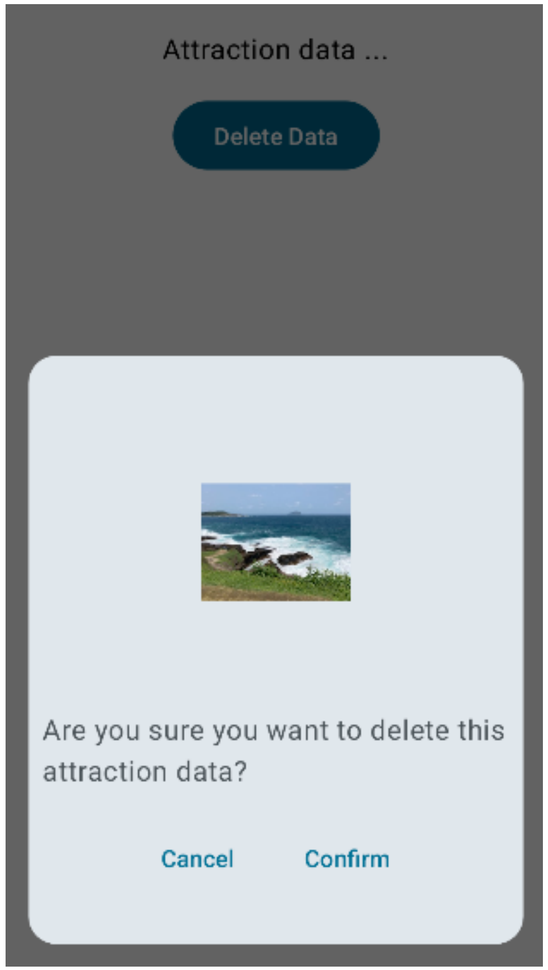
Dialog 自訂對話視窗

黃彬華編撰

- AlertDialog 屬於簡易、制式化的對話視窗
- 如欲自訂對話視窗內容，呼叫 Dialog()，重要參數如下
  - onDismissRequest: 當使用者欲離開此對話視窗
  - content: 自訂對話視窗內容

範例 CustomDialogDemo

黃彬華編撰



- 點擊 "Delete Data" 按鈕會跳出對話視窗
- 點擊對話視窗上的按鈕會顯示對應訊息

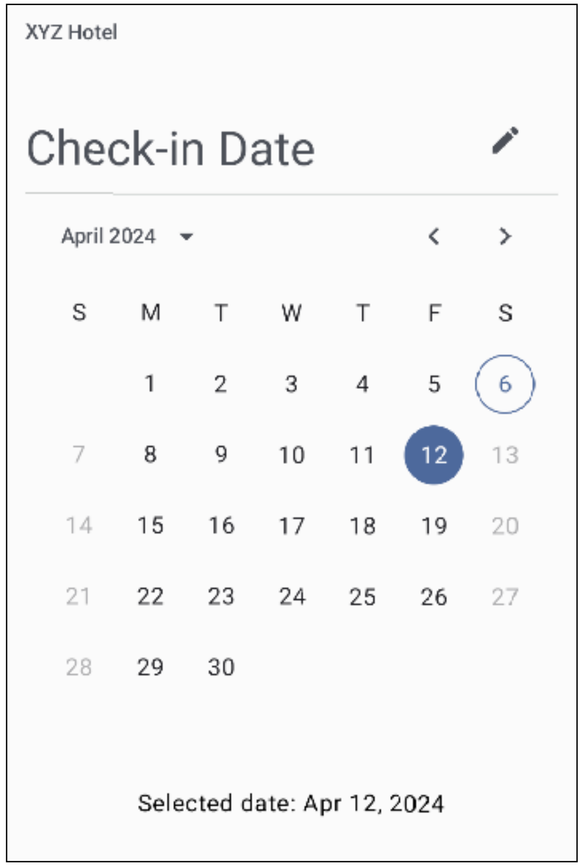
DatePicker 日期挑選器

黃彬華編撰

- DatePicker 可以讓使用者選擇日期，也可切換至日期輸入模式
- 呼叫 DatePicker() 建立，重要參數如下
  - state: 呼叫 rememberDatePickerState() 建立 DatePickerState 物件以設定 date picker 狀態，rememberDatePickerState() 重要參數如下
    - selectableDates: SelectableDates 介面用來限制可選擇的日期與年
  - title: 最上方的標題
  - headline: 日曆上方的標題

範例 DatePickerDemo

黃彬華編撰



- 在 DatePicker 上點選日期，會將選取日期顯示在下方的 Text 上

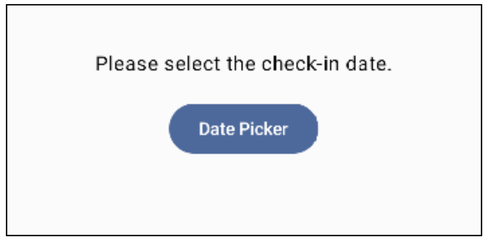
DatePickerDialog 日期挑選視窗

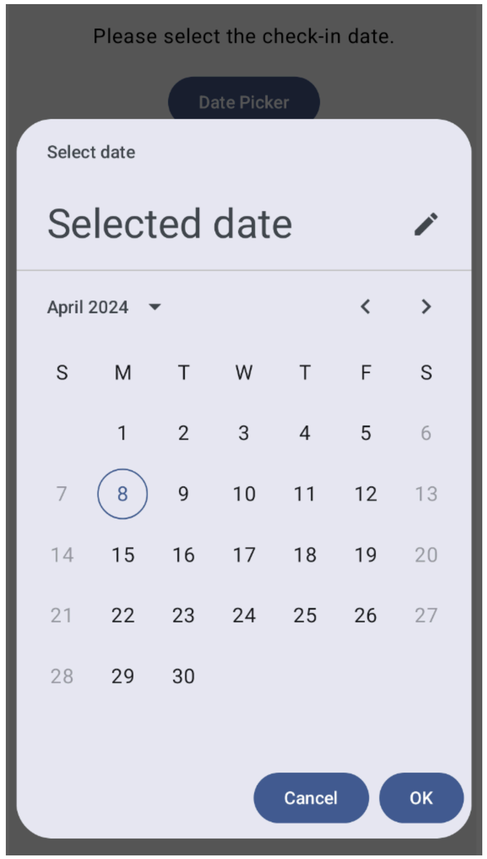
黃彬華編撰

- DatePicker 常會置入對話視窗內，以彈出方式讓使用者選擇日期
- 呼叫 DatePickerDialog() 建立，重要參數如下
  - onDismissRequest: 點擊對話視窗外部或 back 按鈕時呼叫，並非點擊 dismissButton 時呼叫
  - confirmButton: 設定確定按鈕
  - dismissButton: 設定取消按鈕
  - content: 對話視窗內容，例如 date picker

範例 DatePickerDialogDemo

黃彬華編撰

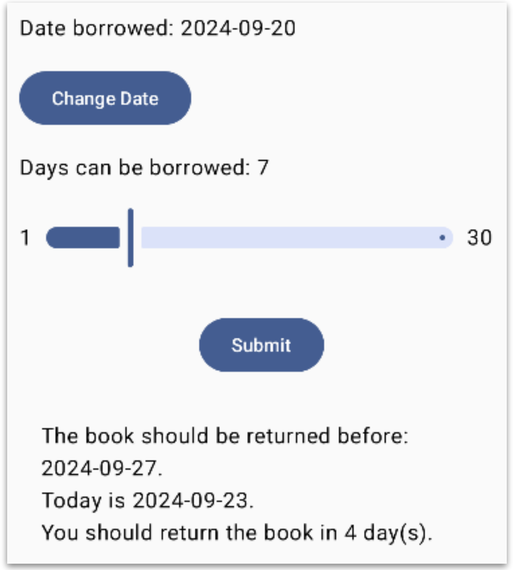




- 點擊 "Date Picker" 按鈕會彈出 DatePickerDialog 對話視窗
- 在 DatePickerDialog 上點選日期後點擊 OK 按鈕會關閉對話視窗，並將選取日期顯示在上方的 Text

Ex 4.2

黃彬華編撰



- 挑選借書日期
  - 日期可挑選範圍：前一個月到今日
- 輸入期限
  - 透過 Slider 選取可借幾天
- 點擊 Submit 按鈕後顯示
  - 還書日期
  - 依據今日是否逾期顯示
    - 未逾期：顯示還有幾日才需要還書
      - 例如：You should return the book in 1 day(s)
    - 今日為還書日：顯示今日需還書
      - 例如：You should return the book today.
    - 已逾期：1 日罰金 $1，顯示逾期天數與罰金總額
      - 例如：This book requires a fine of 2 dollar(s) for a 2-day overdue period.

TimePicker 時間挑選器

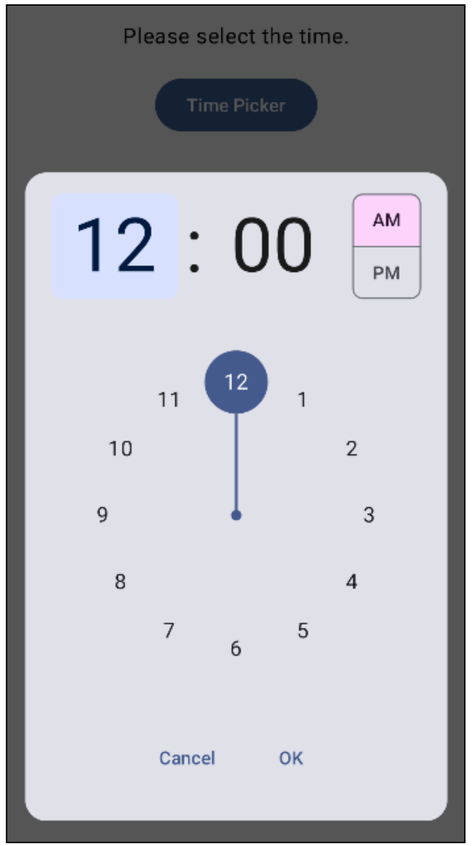
黃彬華編撰

- TimePicker 常會置入對話視窗內，以彈出方式讓使用者選擇時間
  - 函式庫沒有提供 TimePickerDialog，需要自訂 Dialog
- 呼叫 TimePicker() 建立 TimePicker，重要參數如下
  - state: 呼叫 rememberTimePickerState() 建立 TimePickerState 物件以設定 TimePicker 狀態

範例 TimePickerDialogDemo

黃彬華編撰





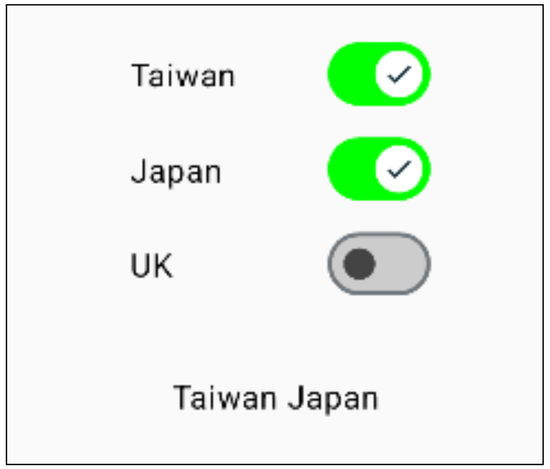
- 點擊 "Time Picker" 按鈕會彈出 TimePickerDialog 對話視窗
- 在 TimePickerDialog 上點選時間後點擊 OK 按鈕會關閉對話視窗，並將選取時間顯示在上方的 Text

Switch 開關選項

黃彬華編撰

- 使用者藉由點擊 Switch 達到開關或選擇功能
- 呼叫 Switch() 建立，重要參數如下
  - checked: 開關狀態
  - onCheckedChange: 開關狀態改變時呼叫
  - thumbContent: 設定圖示
  - colors: 設定顏色

範例 SwitchDemo



- 點按 Switch 來選取欲旅遊的地方，並將選取結果顯示在下方

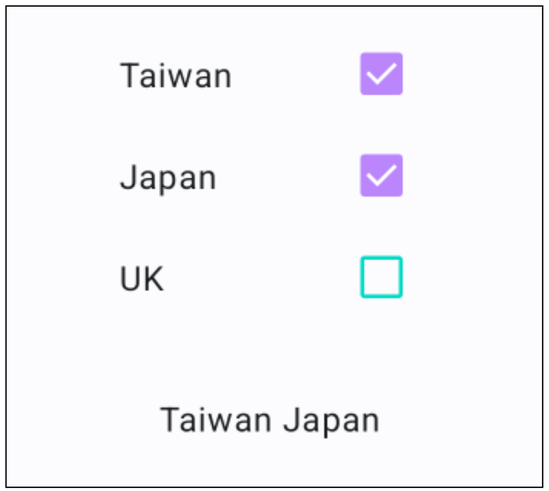
Checkbox 多選功能

黃彬華編撰

- 使用者藉由點擊 Checkbox 達到多選功能
- 呼叫 Checkbox() 建立，重要參數如下
  - checked: 選取狀態
  - onCheckedChange: 選取狀態改變時呼叫
  - colors: 設定顏色

範例 CheckboxDemo

黃彬華編撰



- 點按 Checkbox 來選取欲旅遊的地方，並將選取結果顯示在下方

RadioButton 單選功能

黃彬華編撰

- 使用者藉由點擊 RadioButton 達到單選功能
- 容器元件 (column, row) 加上 Modifier.selectableGroup()
  - 將 RadioButton 群組化方能實現單選功能
- 呼叫 RadioButton() 建立，重要參數如下
  - selected: 選取狀態
  - onClick: 點擊時呼叫
  - colors: 設定顏色

範例 RadioButtonDemo

黃彬華編撰



- 點按 RadioButton 來選取尺寸，並將選取結果顯示在下方

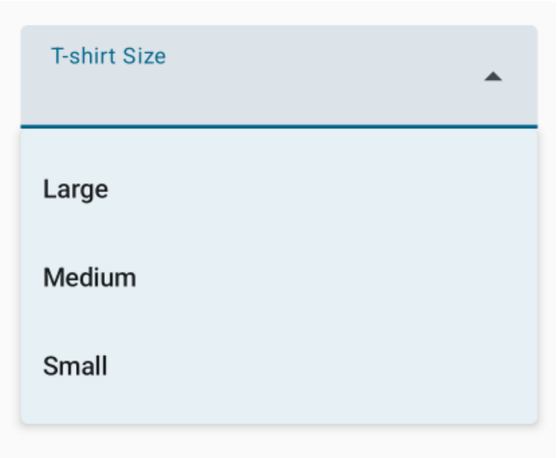
ExposedDropdownMenu 下拉選單

黃彬華編撰

- 使用者藉由點選下拉選單達到單選功能
- 先呼叫 ExposedDropdownMenuBox() 建立容器物件，用來裝 TextField 與 ExposedDropdownMenu 元件
  - 呼叫 TextField() 建立文字方塊
    - readOnly: 設為 true 則無法輸入
    - trailingIcon: 末端顯示下箭頭圖示
  - 呼叫 ExposedDropdownMenu() 建立下拉選單，重要參數如下
    - expanded: 是否彈出下拉選單
    - onDismissRequest: 點擊下拉選單外部
    - content: 建立下拉選單內容；由 DropdownMenuItem 選項元件組成

範例 DropdownMenuDemo

黃彬華編撰



- 點按下拉選單來選取尺寸，並將選取結果顯示在下方 Text
  - 使用者可輸入文字以過濾選項

Ex 4.3

黃彬華編撰

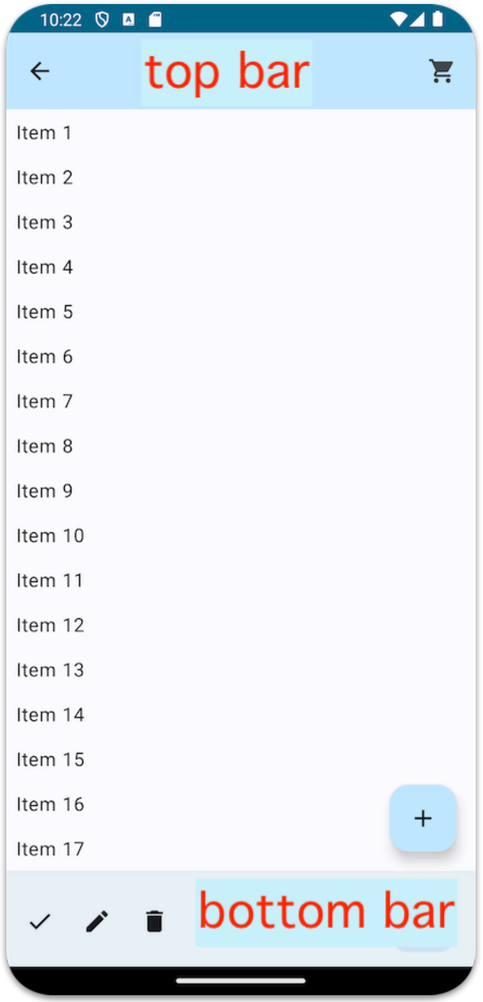


- 建立一個包含四季的下拉選單，使用者選擇季節後會顯示對應的描述。使用者僅能選擇選項，無法輸入文字
  - 春：春暖花開
  - 夏：夏日炎炎
  - 秋：秋高氣爽
  - 冬：冬風凜冽

Scaffold 版面配置容器

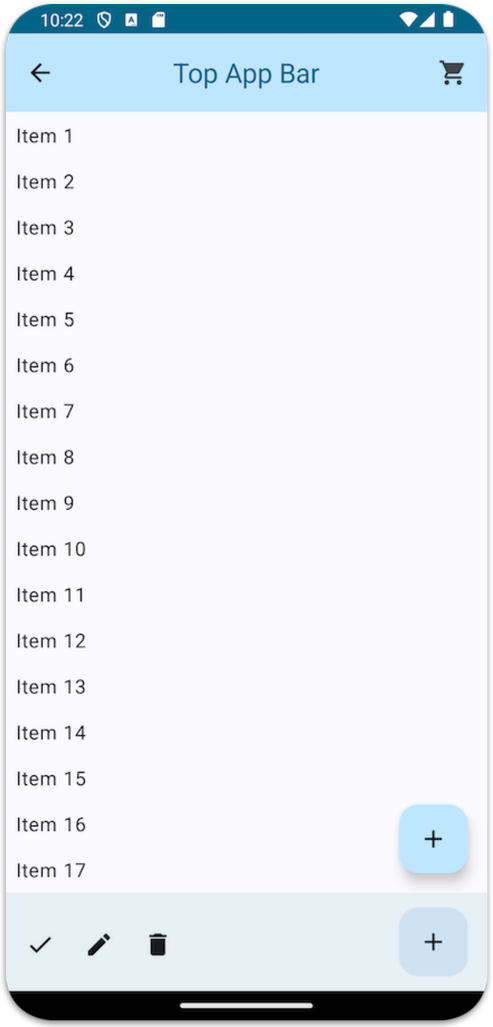
黃彬華編撰

- Scaffold 是一種完整的基礎版面配置容器，可為複雜的使用者介面提供標準化架構，讓應用程式各頁面擁有一致的外觀和風格。它有下列主要屬性可供版面配置：
  - topBar：配置上方列
  - bottomBar：配置下方列
  - snackbarHost：配置 Snackbar
  - floatingActionButton：可在版面右下方配置浮動按鈕



範例 ScaffoldDemo

黃彬華編撰



- 使用 Scaffold 建立整個版面架構
- 點擊任何按鈕都會以 Snackbar 顯示結果

表格

黃彬華編撰

- 表格是以格子狀 (grid) 呈現大量資料
  - 以 List 儲存大量資料，以 LazyVerticalGrid 顯示資料內容
- LazyVerticalGrid
  - lazy 代表只會載入要顯示的項目；尚未顯示的項目則不會載入，以節省記憶體空間
  - 呼叫 items() 可走訪 List 每個項目；再搭配其他 UI 元件，例如 Image, Text 來呈現該項目資料

範例 GridDemo

黃彬華編撰

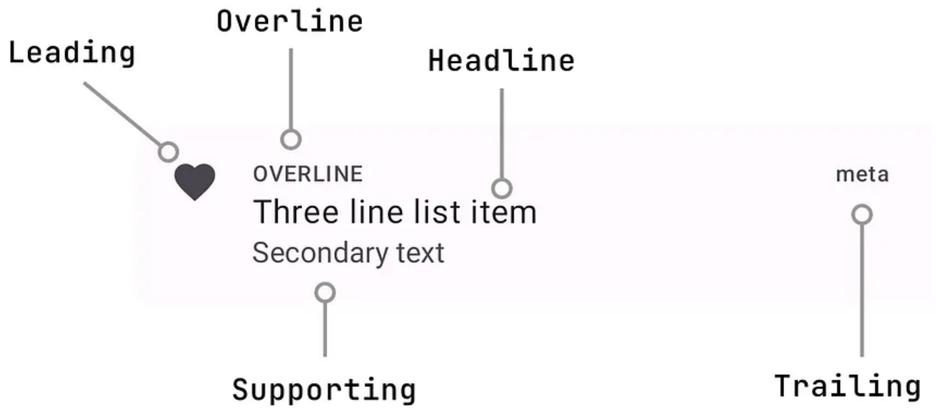


- 以表格方式呈現多本書資訊
- 可以上下捲動
- 點擊任一書本會彈出 Snackbar 顯示該書資訊

列表

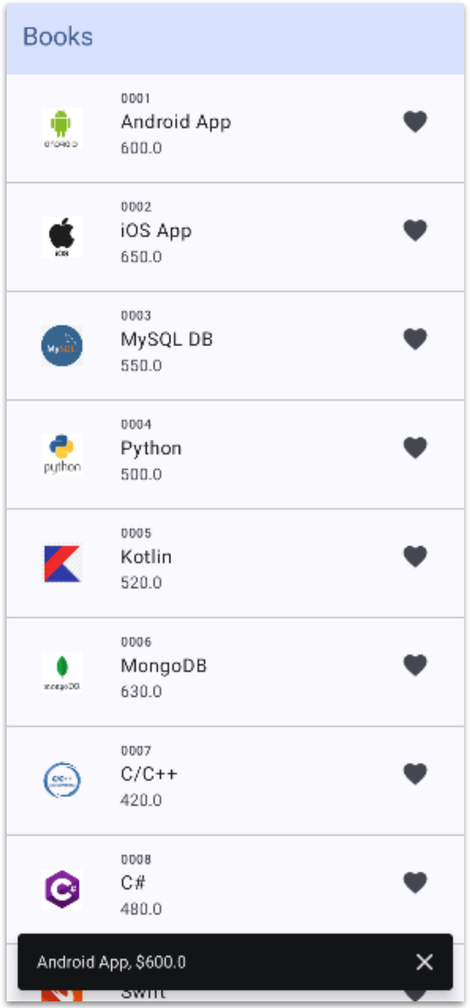
黃彬華編撰

- 列表 (lists) 並非由單一 UI 元件負責；通常是由垂直捲動容器與 ListItem 這兩個元件組成
- 建立垂直捲動容器
  - LazyColumn：只會載入要顯示的項目，以節省記憶體空間，建議搭配列表使用
    - 不可加上垂直捲動功能設定：「modifier = Modifier.verticalScroll(rememberScrollState())」，否則錯誤
  - Column
    - 需要加上垂直捲動功能設定：「modifier = Modifier.verticalScroll(rememberScrollState())」
- ListItem 用來顯示列表內容物，有下列設定，並參看下圖
  - headlineContent, overlineContent, supportingContent, leadingContent, trailingContent



範例 ListsDemo

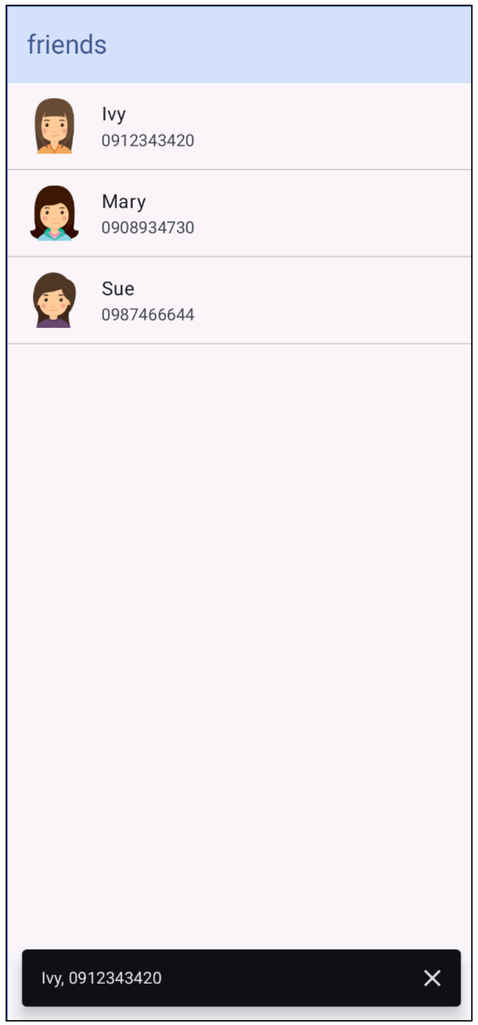
黃彬華編撰



- 以列表方式呈現多本書資訊
- 可以上下捲動
- 點擊任一書本會彈出 Snackbar 顯示該書資訊

Ex 4.4

黃彬華編撰



- 建立一個 app，以列表方式呈現所有好友資訊
  - 好友資訊包含好友照、名稱、電話
- 點擊任一好友會彈出 Snackbar 顯示該好友名稱與電話

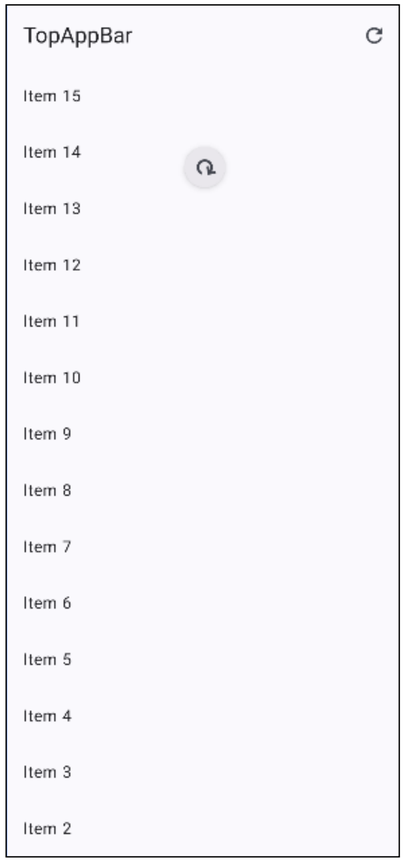
下拉更新

黃彬華編撰

- 下拉更新 (pull to refresh) 提供使用者以下滑手勢對可垂直捲動元件做一個更直覺的更新內容方式
- 使用步驟
  - 呼叫 rememberPullToRefreshState() 建立 PullToRefreshState 物件以儲存下拉狀態
  - 建立 PullToRefreshBox 容器
    - 設定 user 使用下拉手勢時的 callback 方法
    - 並在其內建立 LazyColumn 以呈現資料內容

範例 PullToRefreshDemo

黃彬華編撰



- 每次向下拖曳，就會增加 5 個項目

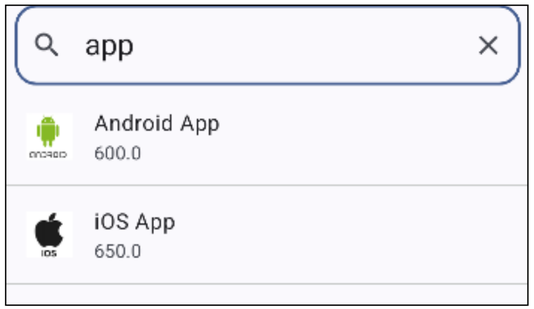
列表搜尋

黃彬華編撰

- Scaffold 標題列加上 OutlinedTextField 讓使用者輸入關鍵字
- 利用篩選結果建立新的 List 集合
- 將搜尋結果的 List 以 ListItem 呈現

範例 ListsSearchDemo

黃彬華編撰



- 輸入欲搜尋的關鍵字後，會將搜尋結果顯示在下方
- 點擊任一書本會彈出 Snackbar 顯示該書訊息

列表的增刪改查

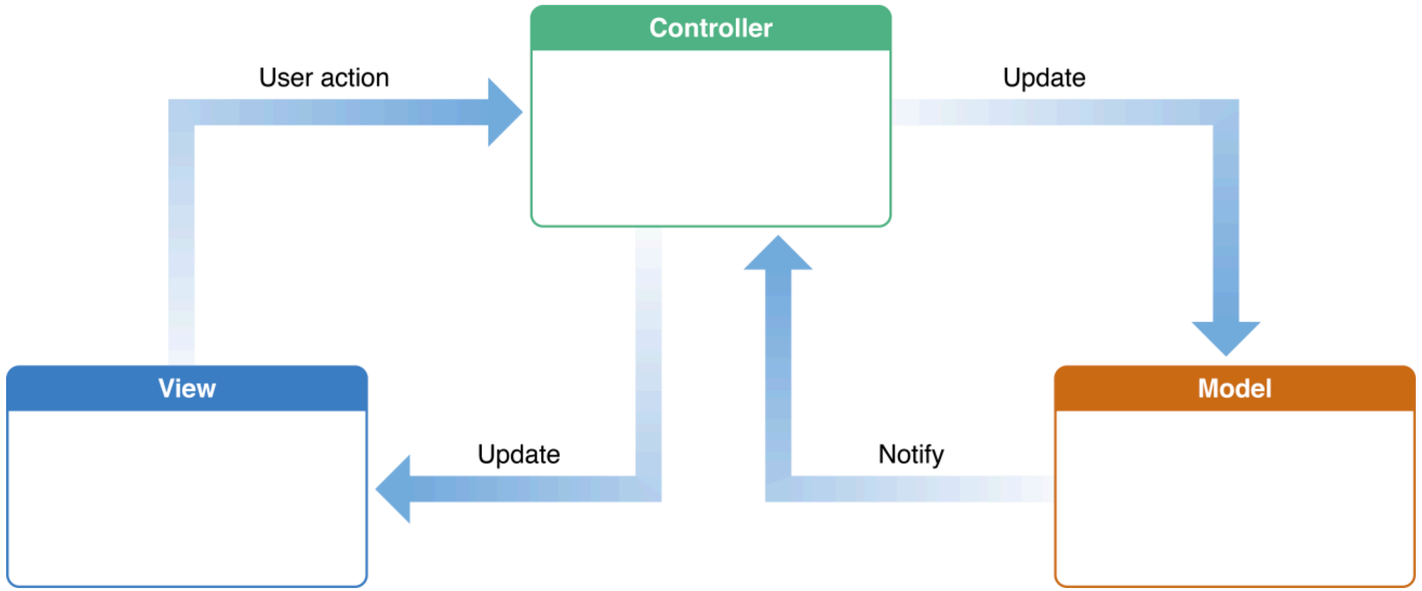
黃彬華編撰

- 建立自訂 ViewModel 類別，並在其內建立：
  - MutableStateFlow
    - 用來監控指定資料狀態，當資料一改變即可通知對應畫面更新
    - MutableStateFlow 常與 ViewModel 搭配，可以讓 UI 元件在生命週期期間作出適當更新
  - 增刪改查對應方法

MVC 模式

黃彬華編撰

- MVC (Model View Controller) 是傳統設計模式 (design pattern)，分成下列3個部分
  - Model：處理商業邏輯與資料存取部分。Room 屬之
  - View：圖形化使用者介面。舊 layout 檔案屬之
  - Controller：擔任上述2個部分的溝通橋樑。activity 屬之



MVC 走樣了？

黃彬華編撰

- 既然 controller 擔任 view 和 model 之間溝通橋樑，就意味著既要處理 view 的變化，也要隨時向 model 要資料，工作份量頗為吃重
- server 端平台開發上，已經有許多框架設計分攤了controller 的工作，但 Android app 原本框架設計讓 controller 扛重擔，而被戲稱為 MVC (Massive View Controller) 架構
- controller 過於肥大的結果就是造成難以維護，甚至牽一髮而動全身
- 既然 controller 負擔過重，那就找人來幫忙分攤重擔，於是 view model 出現了

MVVM 模式

黃彬華編撰

- MVVM (Model View ViewModel) 這種設計模式簡單來說就是新增了一個 ViewModel 來分攤 Controller 的重擔
- 從此 ViewModel 負責與 Model 聯繫以存取資料，資料一旦更新還需告知 Controller，然後再由 Controller 更新 View 的畫面
- 如果再搭配 Android 的狀態變數就可做到單向 / 雙向 (one- / two-way) 的 data binding，那麼 Controller 的負擔又更輕了



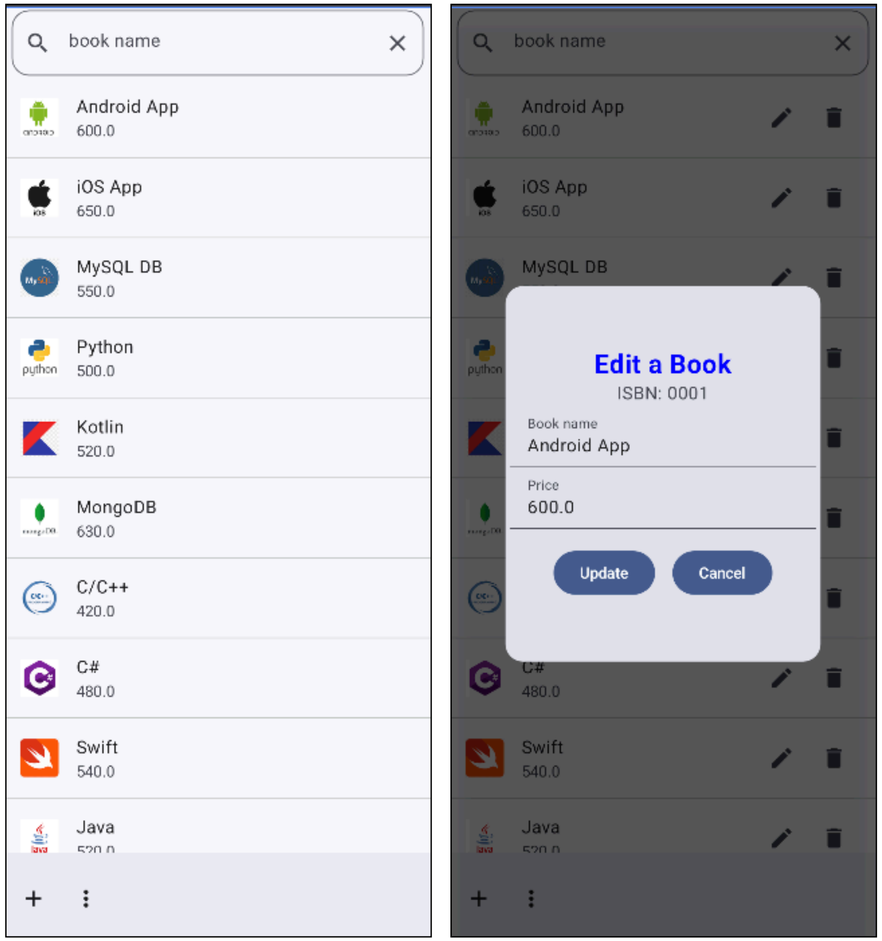
MVVM 各元件說明

黃彬華編撰

- Model
  - 建立資料模型 (data model)，例如：建立商品類別與對應屬性
  - 建立資料存取方法，例如：商品的增刪改查
  - 實作：使用 Room 框架
- View (與 Controller)
  - 方便使用者輸入與回應結果給使用者
  - 實作：使用 composable 函式建立 UI 畫面
- ViewModel
  - 宣告與商業邏輯有關的資料，例如：建立商品列表
  - 建立方法以處理使用者與上述資料間的互動，例如：產生訂單
  - 實作：建立類別並繼承 androidx.lifecycle.ViewModel

範例 ListsCRUDDemo

黃彬華編撰



- 一開始會列出所有書籍
- 最上方可輸入欲查詢的書名
- 點擊左下角「+」按鈕會跳出新增對話視窗
- 點擊左下角「...」按鈕會在每列顯示編輯與刪除的圖示按鈕
  - 點擊編輯按鈕會跳出編輯對話視窗
  - 點擊刪除按鈕會直接刪除該筆資料
- Preview 如果無法支援此範例所有操作功能，請改用**模擬器**